RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

UNIT- III (Inheritance & Polymorphism)

- Types of inheritance
- Constructors in derived and base class
- Abstract classes
- Compile and run time Polymorphism
- Role of Constructors in inheritance
- Concept of Polymorphism
- Overriding Super Class Methods
- Use of “super” keyword
- Implementing interfaces
- Dynamic method dispatching by down-casting and up-casting.

Inheritance

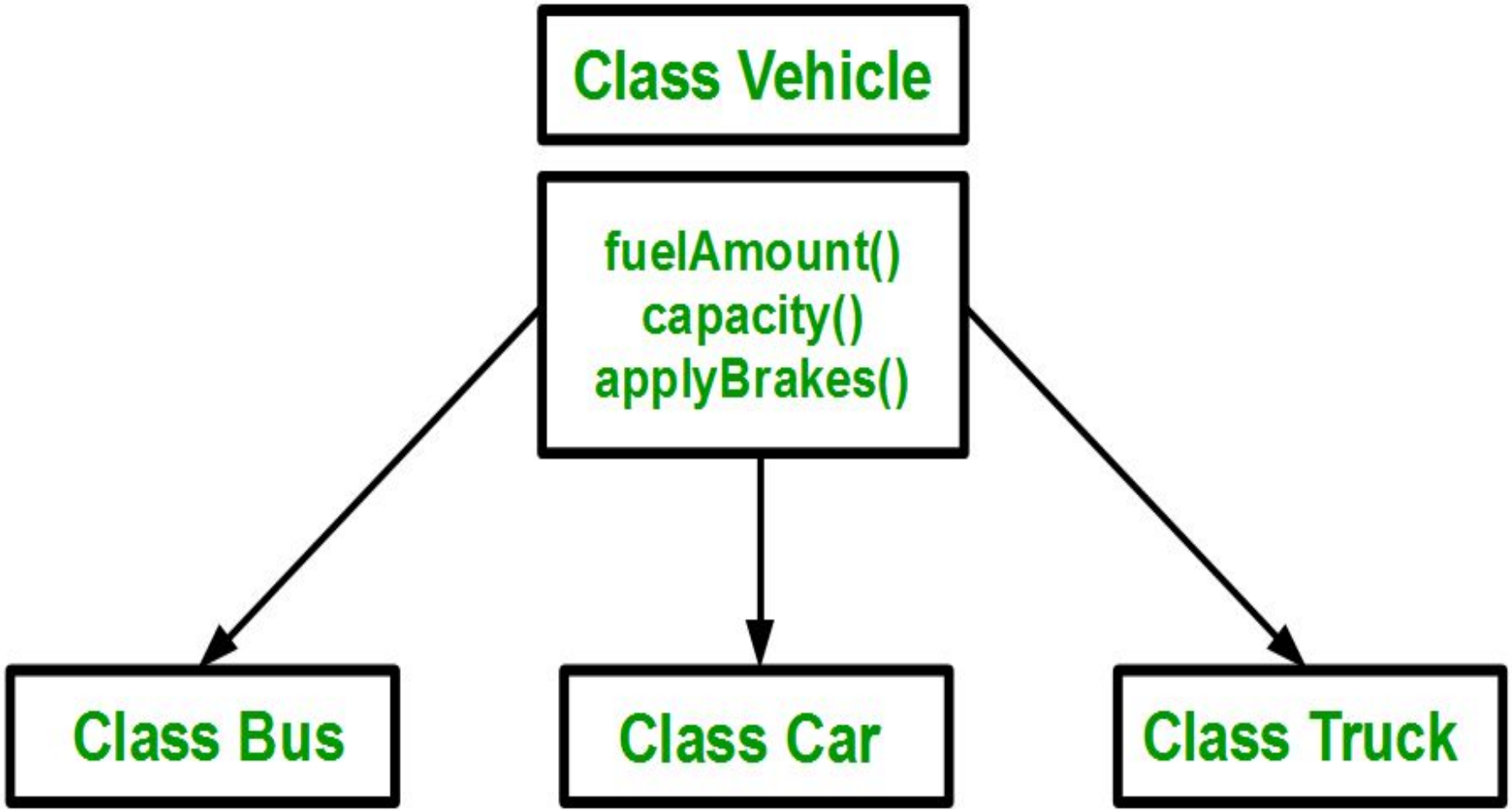
- Inheritance is the process by which objects of one class acquire the properties of another class.
- Inheritance provides the idea of reusability i.e., code once written can be used again and again in number of new classes.
- This mean that we can add additional features to an existing class with out modifying it.
- The new class can use all or some of the features of the already existing class and can define his own members too.

Inheritance

- It is the mechanism of deriving a new class from an already existing class.

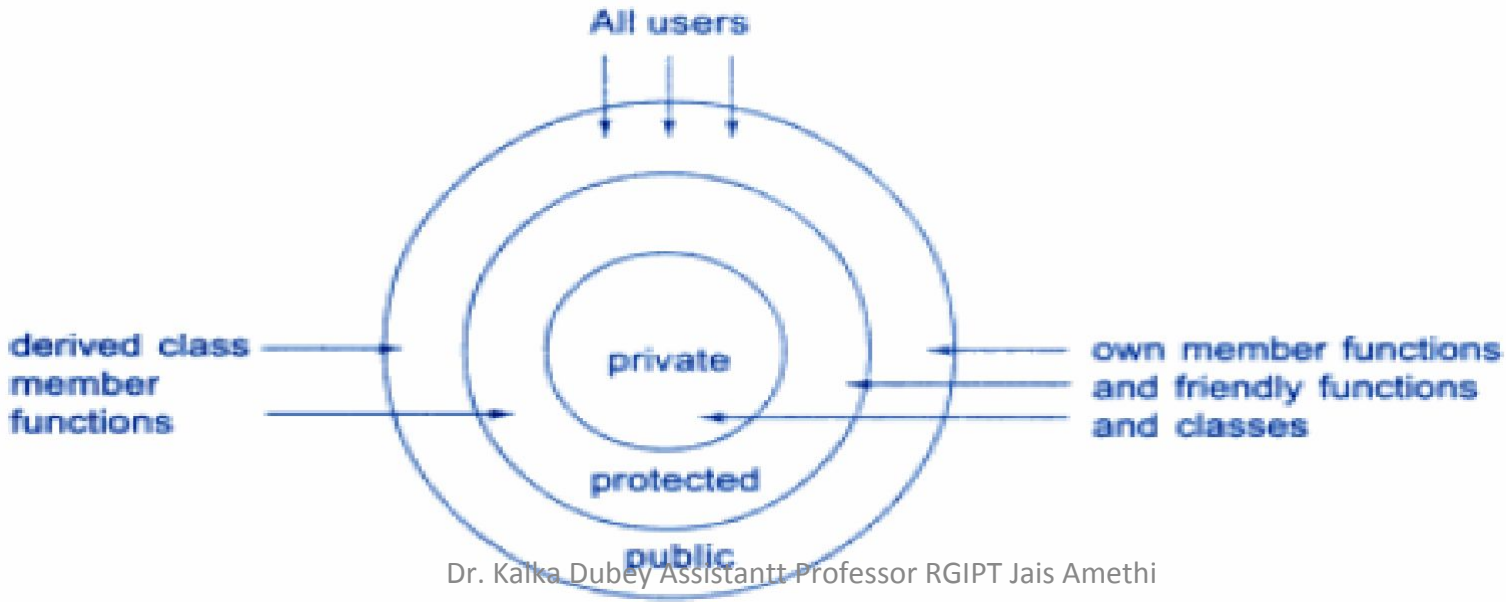
- The old class and new class is called by the name-pair base-derived, parent-child, super-sub etc.

Inheritance



Inheritance

Base class visibility	Derived class visibility		
	Public derivation	Private derivation	Protected derivation
Private ⟶	Not inherited	Not inherited	Not inherited
Protected ⟶	Protected	Private	Protected
Public ⟶	Public	Private	Protected



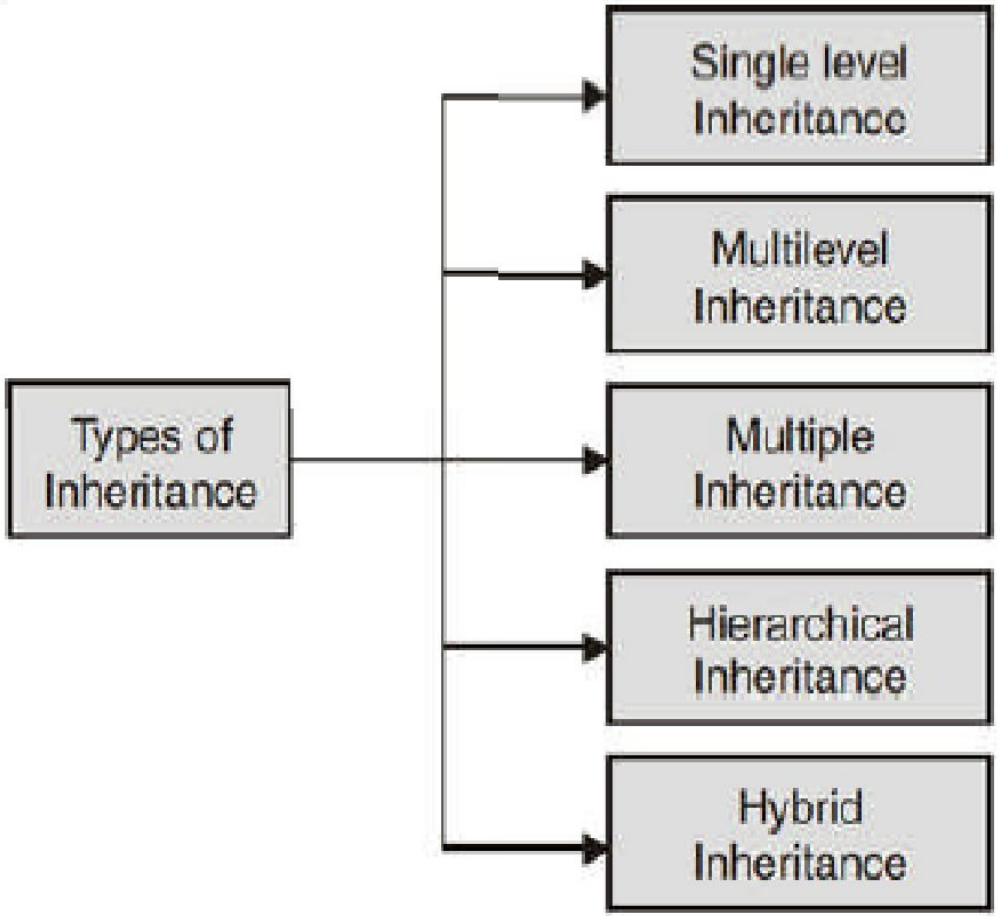
Types of Inheritance

- In general inheritance is of five types:

1. Single level inheritance 
2. Multiple inheritances 
3. Multilevel inheritance 
4. Hybrid Inheritance 

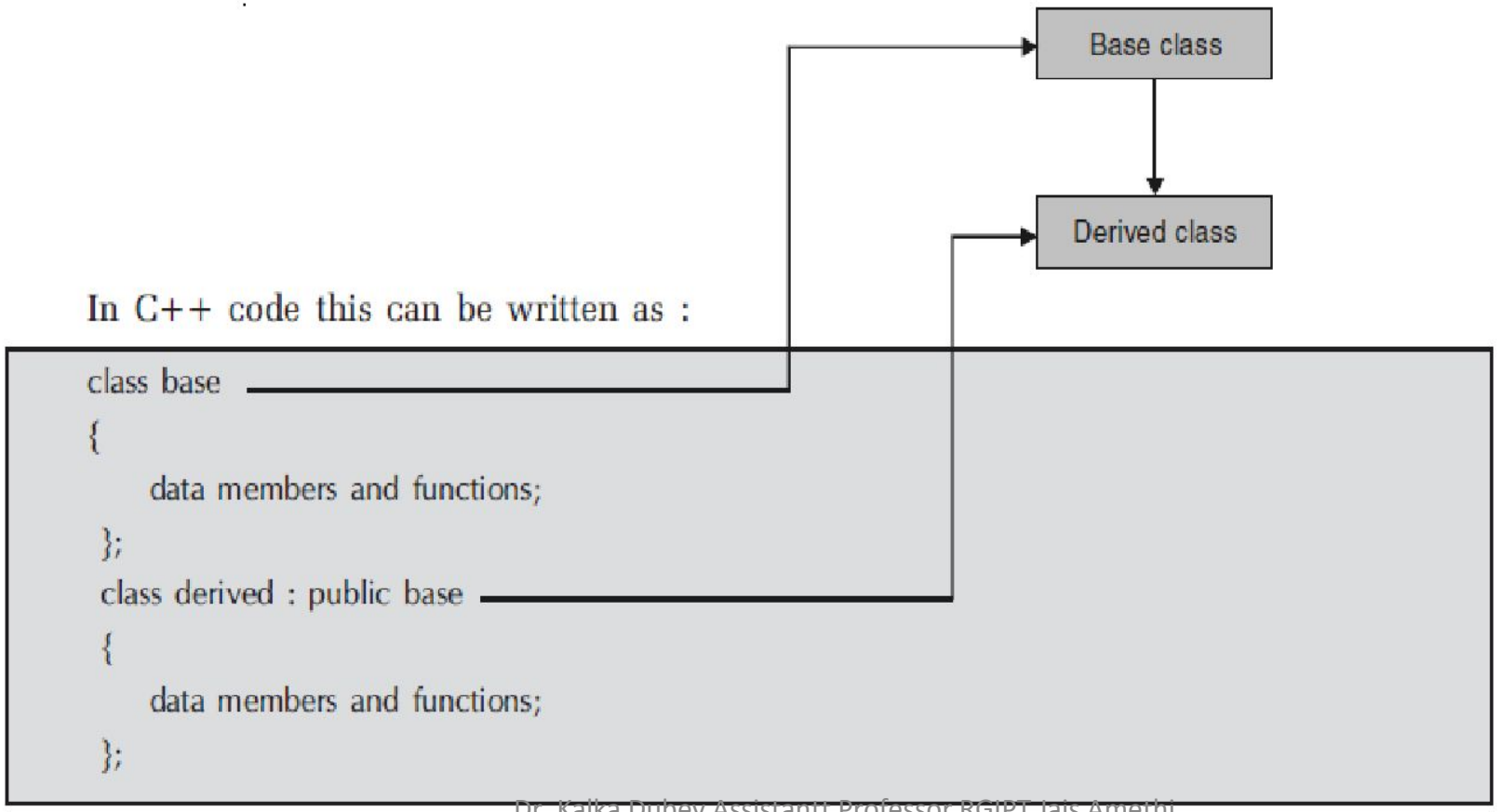
5. Hierarchical inheritance 

Types of Inheritance



Single Level Inheritance

In single level inheritance we have just one base class and one derived class. It is represented as.



In C++ code this can be written as :

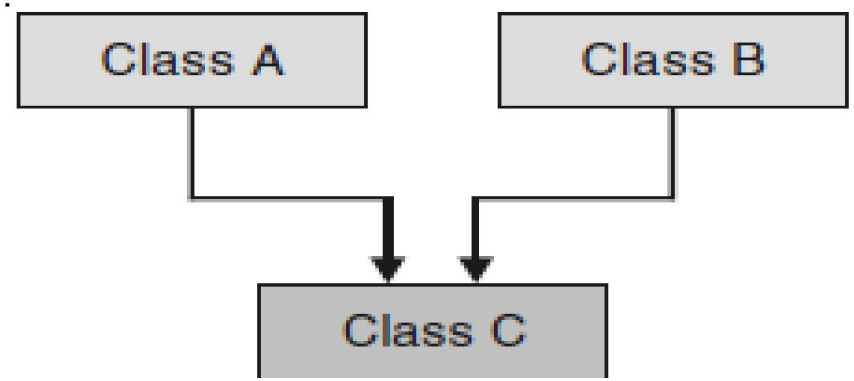
```
class base
{
    data members and functions;
};
class derived : public base
{
    data members and functions;
};
```

Multiple Inheritance

In multiple inheritance a child can have more than parent i.e., a child can inherit properties from more than one class. Diagrammatically this is as shown below

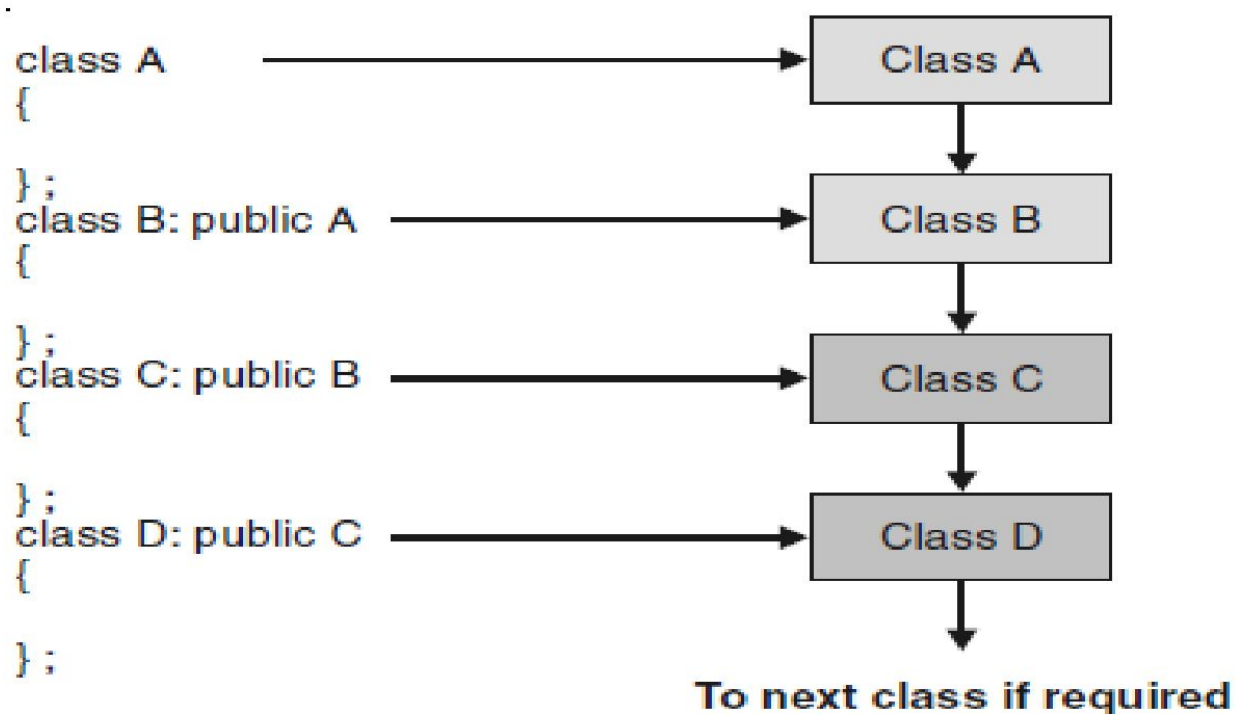
code ⇒

```
class A { };
class B { };
class C : Public A, Public B { };
```



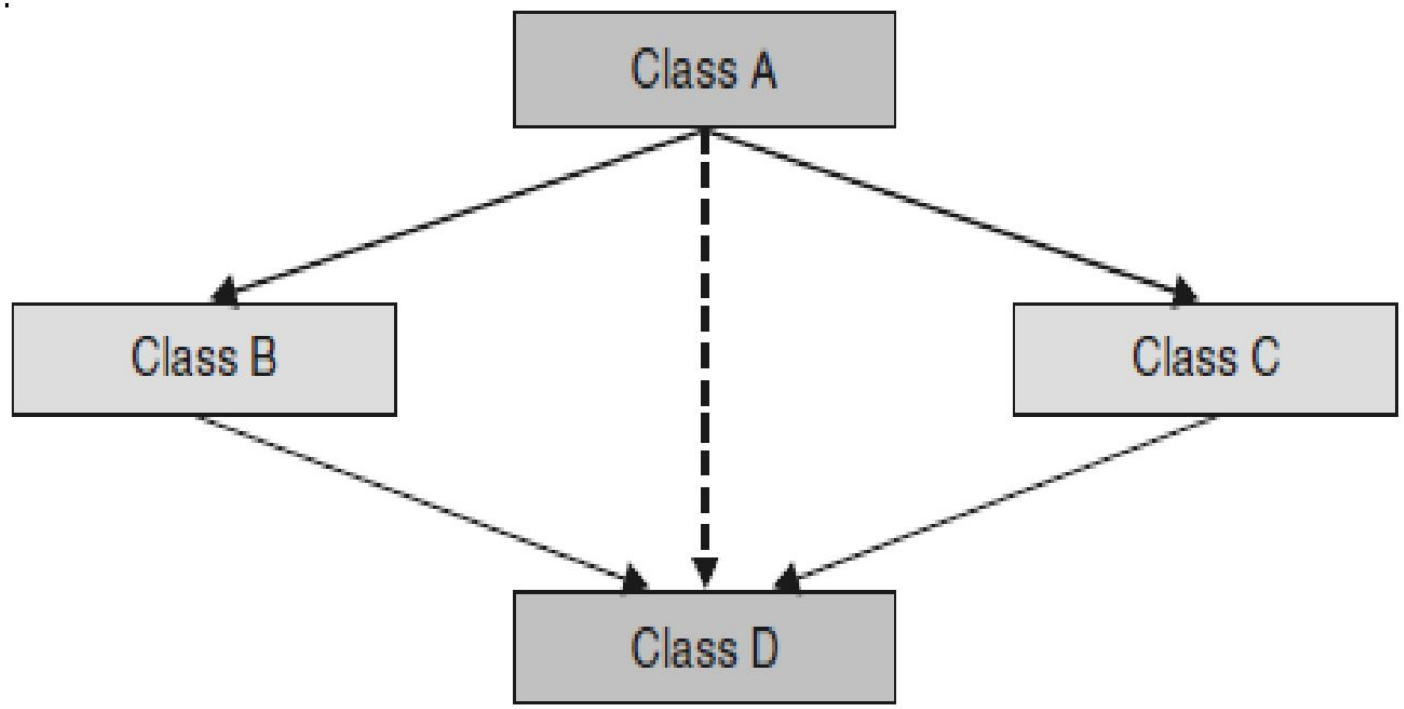
Multilevel Inheritance

In multiple inheritances we have one base class and one derived at one level. At the next level the derived class becomes base class for the next class and so on



Hybrid Inheritance

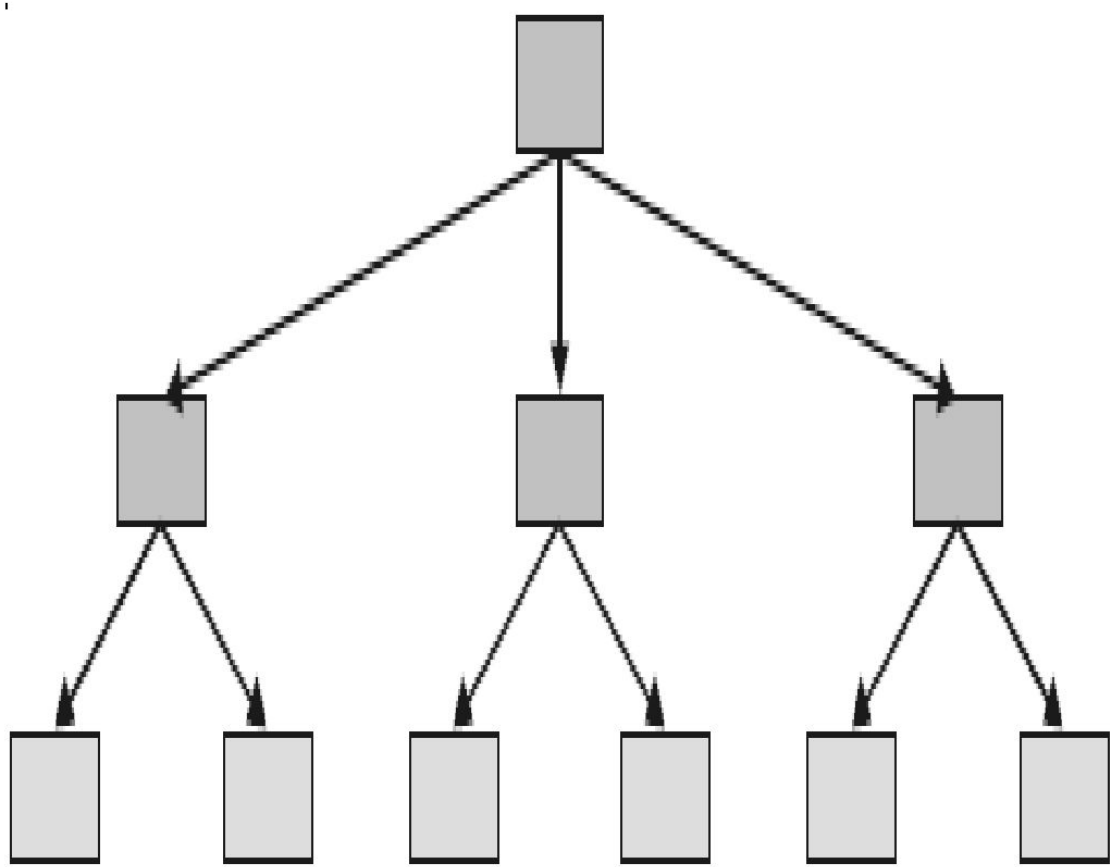
- In Hybrid inheritances we have one base class and one or more derived at one level. At the next level the derived class becomes base class for the next class and so on



Hierarchical Inheritance

- In this type of inheritance multiple classes share the same base class. That is number of classes inherits the properties of one common base class.
- The derived classes again may become base class for other classes.
- For example, a university has number of colleges under its affiliation. Each college may use the university name, the chairperson name, its address, phone number etc.
- There are number of properties or features which a vehicle posses.
- The common properties of all the vehicle may be put under one class vehicle and different classes like two-wheeler, four-wheeler and three-wheeler can inherit the vehicle class

Hierarchical Inheritance



Public, private and protected inheritance

Public Inheritance:

- class B : public A tells the compiler that we are inheriting class A in class B in public followings :

(a) All the public members of class A becomes public members of class B.

(b) All the protected members of class A becomes protected members of class B.

(c) Private members are never inherited.

```
class B : public A
{

};
```

Public, private and protected inheritance

Private Inheritance

class B : private A tells the compiler that we are inheriting class A in class B in private mode. In private mode inheritance note the following points :

(a) All the public members of class A becomes private members of the class B.

(b) All the protected members of the class A becomes private members of class B.

(c) Private members are never inherited.

```
class B : private A
{

};
```

Public, private and protected inheritance

Protected Inheritance:

- class B : protected A tells the compiler that we are inheriting class A in class B in protected mode. In protected mode inheritance note the following points :

(a) All the public members of class A becomes protected members of class B.

(b) All the protected members of class A becomes protected members of class B.

(c) Private members are never inherited.

```
class B : protected A
{

};
```

Example 1: Demo of single level inheritance

```
#include <iostream.h>
using namespace std;
class super
{
public :
void sup_show( )
{
cout<<“Hello from show of super”<<endl;
}
};
class sub :public super
{
};
void main( )
{
sub o1;
o1.sup_show ( );
}
```



Example 2: Demo of single level private inheritance

```
#include <iostream.h>
using namespace std;
class super
{
protected :
int num;
void input(int x)
{
num = x;
}
};
class sub :super
{
public :
void sub  show( )
```



Output

OUTPUT :

Compile Time Error.

Example 3: Demo of single level protected inheritance

```
#include <iostream.h>
using namespace std;
class super
{
public :
void show( )
{
cout<<“hello from super”<<endl;
}
};
class sub :protected super
{
};
void main( )
```

Output

*ERROR :’*show*’ cannot access public member declared in class ‘super’*

Example 4: Finding maximum of three class's data (Multilevel)

```
#include <iostream.h>
#include <conio.h>
class first
{
protected :
int fa;
public :
void input_f( )
{
cout<<"Enter the value for fa :=";
cin>>fa;
}
};
class second :public first
{
protected :
```

Example 5: Finding maximum of three class's data

```
int max( )
{
int t1,t2;
t1=fa>sa ?fa :sa;
t2=ta>t1 ?ta :t1;
return t2;
}
};
void main( )
{
clrscr( );
third t;
t.input_t( );
t.show( );
cout<<"Max is "<<t.max( )<<endl;
getch( );
```

Output

Enter the value for fa : =123

Enter the value for sa : =567

Enter the value for ta : =234

Data of class first fa : =123

Data of class second sa : =567

Data of class third ta : =234

Max is 567

Exercise 1: Create two Classes named as Base and exponent which further drive a new class power using the multiple inheritances and finding the power from base and exponent classes data members and member function. Take all the input from the users.

Finding power from base and exponent (Multiple)

```
#include <iostream.h>
#include <conio.h>
#include <stdlib.h>
class base
{
protected :
int ba;
public :
void input_base( )
{
cout<<“Enter the base :=”;
cin>>ba;
}
void show_base( )
{
cout<<“base :=”<<ba<<endl;
```

Finding power from base and exponent

```
void show( )
{
show_base( );
show_exp( );
int i;
po=1;
for(i=1;i<=exp;i++)
po=po*ba;
cout<<“power :=”<<po<<endl;
}
};
void main( )
{
power o1;
o1.input( );
o1.show( );
```

Output

Enter the base : =5

Enter the exponent : =3

base : =5

exponent=3

power : =125

Example:

```
#include<iostream>
using namespace std;
class A {
   public:
   void func() {
      cout << " I am in class A" << endl;
   }
};
class B {
   public:
   void func()
{
      cout << " I am in class B" << endl;
   }
};
class C: public A, public B
```

Resolving Ambiguity

- In multiple inheritance when two base classes having same name for their functions and data, ambiguity arises when they are used in the derived class.
- For example, consider the two classes A and B having same function signature for function named show like void show ( ).
- Both the classes are inherited in the third class C. Now how class C can use show of A class and show of B class individually.
- Simply accessing show function in class C creates confusion as to which show we want to refer; show of class A or show of class B.

Resolving Ambiguity

- In these situations we can use the scope resolution operator with class names to functions for accessing the function of individual class.

o1.A::show();

C o1;

o1.A : :show( ); //calls show of A class

o1.B : :show( ); //calls show of B class

o1.show( ); // calls show of C class

Example 7: Resolving ambiguity in multiple inheritance

```
#include <iostream>
class A
{
protected :
int num;
public :
void show( )
{
cout<<“show of A\n”;
cout<<“num of A=”<<num<<endl;
}
}
class B
{
protected :
int num;
```

Output

show of A

num of A=20

show of B

num of B=30

show of C

num of C=40

Example 8: Demo of hierarchical inheritance

```
#include <iostream.h>
#include <string.h>
class University
{
protected :
char uname[40];
public :
University( )
{
strcpy(uname,” Ministry of Education”);
}
};
class college1 :public University
{
char cname[50];
public :
college1( )
```

Output

College Name : =ABC

Affiliated to : =Ministry of Education

College Name =DEF

Affiliated to : = Ministry of Education

Example 9: Demo of Hybrid

```
#include <iostream.h>
class A
{
public :
int a;
A( )
{
a=100;
}
};
class B :public A
{
};
class C : public A
{
};
```

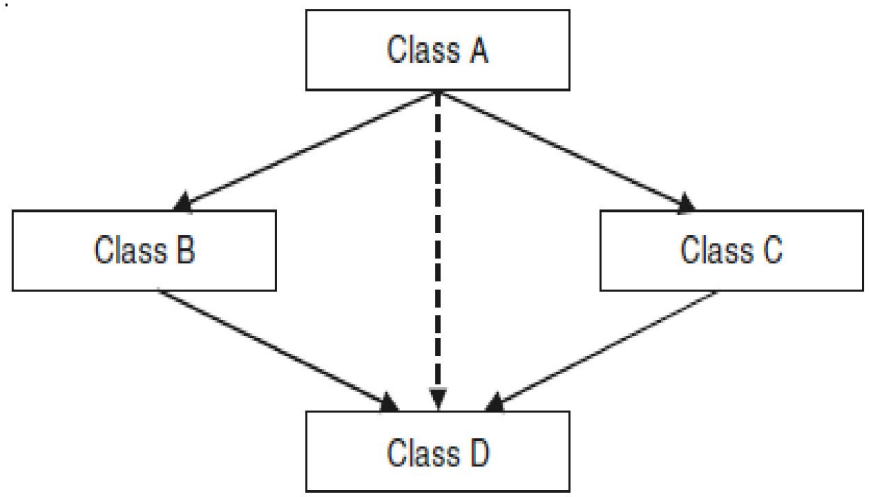
Virtual base class

- Consider the situation where we have one class A. This class A is inherited their features in two other classes B and C.

- Both these classes are inherited in a new class D.

```
class A
{
};
class B :public A
{
};
class C : public A
{
};
class D :public C, public B
{
};
```

Virtual base class



- The data members and member functions of class A are inherited twice to class D. One through class B and second through class C.
- When any data members and member function of class A is accessed by an object of class D, ambiguity arises as to which data and members function would be called ?

Virtual base class

- To resolve this ambiguity when class A is inherited in both class B and class C, it is declared as virtual base class by placing the keyword virtual as :
- When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members. A single copy of its members is shared by all the base classes that use it as a virtual base.

```
Class B : public virtual A
{
};
class C : public virtual A
{
};
```

class B : public virtual A {};

Example 9: Demo of virtual base class

```
#include <iostream.h>
class A
{
public :
int a;
A( )
{
a=100;
}
};
class B :public virtual A
{
};
class C :virtual public A
{
};
```

Output

$$a=100$$

Example 10: Demo of virtual base class 2

```
#include <iostream.h>
class A
{
public :
void show( )
{
cout<<“hello from show of A”<<endl;
}
};
class B :public virtual A
{
};
class C :virtual public A
{
};
```

Output

hello from show of A

Constructor and destructor in inheritance

- When constructors are present both in base and derived classes then how they are called, how values are passed from derived class to base class?

- Assume a small example of single level inheritance in which class A is inherited by class B. Both the classes have their default constructors.

- When an object of class B is created, it calls the constructor of class B, but as this class B has got A as its parent class, **constructor of class A will be called first, then constructor of class B.**

- Now without calling the constructor of base class, data members of base class, **unexpected results may follow**.

Constructor and destructor in inheritance

- Destructor in inheritance: **Destructor of derived class is called first and then destructor of base class is called.**

- So, destroying them in base class has an effect of working with the data members that no longer exist.

- Compiler won't allow this. That's why destructor class call as first so that it can do its works finishing with any of the data members of class base and do its own cleaning.

Example 11: Demo of constructor in single level inheritance

```
#include <iostream.h>
class first
{
public :
first( )
{
cout<<“Hello from con of first”<<endl;
}
};
class second : first
{
public :
second( )
{
cout<<“Hello from con ofsecond”<<endl;
}
```

Output

Hello from con of first

Hello from con of second

Example 12: Constructor & destructor in single level inheritance

```
#include <iostream.h>
class first
{
public :
first( )
{
cout<<“Hello from con”<<endl;
}
~first( )
{
cout<<“Bye Bye from des of first “<<endl;
}
};
class second : public first
{
public :
```

Output

Hello from con

Hello from con of second

Bye Bye from des of second

Bye Bye from des of first

Example 13: Constructor in multilevel inheritance

```
#include <iostream.h>
class first
{
public :
first( )
{
cout<<“Hello from con of first”<<endl;
}
};
class second :public first
{
public :
second ( )
{
cout<<“Hello from con of second “<<endl;
}
```

Output

Hello from con of first

Hello from con of second

Hello from con of third

Example 14: Constructor in multiple inheritance

```
#include <iostream.h>
class first
{
public :
first ( )
{
cout<<“Hello from con first”<<endl;
}
};
class second
{
public :
second( )
{
cout<<“Hello from con of second”<<endl;
}
```

```
class third : public first, public second {
public:
    third( ){
        cout<<" Hello ..... "<<endl;
    }
};
void main( ){
    third obj;
}
```

जिस order में लिखा है उसी order में print होगा।

Output

Hello from con first

Hello from con of second

Hello from con of third

Example 15: Demo of parameterized constructor in inheritance

```
#include <iostream.h>
#include <conio.h>
class first
{
int fa;
public :
first (int x)
{
fa = x;
cout<<“Con of first called”;
}
void fshow( )
{
cout<<“fa =”<<fa<<endl;
}
};
```

constructor of 2nd class derived from first with argument. ←

```
second(int x, int y): first(x) {
#code
}
```

Output

Con of first called

Con of second called

fa =10

sa =20

Pointer to objects

- Similar to pointers to built-in data types we can create pointers to object of class. To create a pointer to an object of class demo we write

demo *ptr;

- Which creates a pointer of type demo class type. Now for an object say d of demo class declared as

demo d

- We can store address of this object into pointer ptr as :

ptr = &d;

- Now any data member or function of demo class can be accessed using pointer as

ptr->func_name( );

ptr->data_member;

Example 1: Demo of pointer object

```
#include <iostream.h>
class demo
{
public :
void show( )
{
cout<<“Hello from show”<<endl;
}
void bye_bye( )
{
cout<<“BYE BYE”<<endl;
}
};
void main( )
{
demo d;
```

Output

OUTPUT :

Hello from show

BYE BYE

Virtual Function

- A virtual function is a member function which is declared within a base class and is re-defined (overridden) by a derived class.
- It is a function that is declared virtual by placing the keyword virtual before the function definition/declaration.
- The derived classes as per its requirements override the virtual function and provide the virtual coding specific to that class only.

Virtual Function

- The derived class is free to decide whether it wants to override the virtual function defined in the base class or not.

- Virtual function behaves like any other member function but they show their importance when accessed via pointers of base class.

- They are the basis for implementing runtime polymorphism in C++.

Rules for Virtual Function

- The rules are :

1. Virtual function cannot be declared outside the class i.e., must be member of some class.
2. Virtual function can be called using object pointers and even using objects but actual works can be seen only with pointers.
3. Virtual function cannot be declared as static.
4. Virtual constructor is not possible but we can have virtual destructor.
5. A virtual function can be friend to another class.
6. Making a virtual function in base class is simply a choice for the derived class.
7. The derived class may or may not override virtual function of the base class.
8. In the absence of virtual keyword, the function to be invoked is determined by the type of pointer.
9. The compiler does not check what type of object is stored in the pointer.

Rules for Virtual Function

10. When virtual keyword is present, functions are called on the basis of which type of object pointer hold in it.

11. Never use ++ or − − operator on pointer of base class pointer to get the next object of derived class. It will only forward/backward the pointer relative to its own class type.

12. Pointers follow the hierarchy of inheritance and pay regards to the older.

Example 1: Demo of Virtual function

```
#include <iostream>
class first
{
public :
virtual void show( )
{
cout<<“Hello from show first class”<<endl;
}
void display( )
{
cout<<“Hello from display of first class”<<endl;
}
};
class second :public first
{
};
```

* A Base class pointer can point to a derived class pointer.

Output

Hello from show first class

Hello from display of first class

Hello from show first class

Hello from display of first class

Example 2: Demo of virtual function

```
#include <iostream.h>
class first
{
public :
virtual void show( )
{
cout<<“Hello from show of first class”<<endl;
}
virtual void display( )
{
cout<<“Hello from display of first class”<<endl;
}
virtual void fun( )
{
cout<<“Hello from fun of first”<<endl;
}
```

Example 1: Demo of virtual function

```
void main( )
{
first *ptr;
first f;
second s;
third t;
ptr= &f;
ptr->show( );
ptr->display( );
ptr->fun( );
ptr=&s;
ptr->show( );
ptr->display( );
ptr->fun( );
ptr=&t;
ptr->show( );
```

Output

Hello from show of first class

Hello from display of first class

Hello from fun of first

Hello from show of second class

Hello from display of first class

Hello from fun of second

Hello from show of first class

Hello from display of third class

Hello from fun of third class

Example 3: Demo of virtual function

```
#include <iostream.h>
class first
{
public :
virtual void show( )
{
cout<<“Hello from show of first class”<<endl;
}
};
class second :public first
{
public :
void show( )
{
cout<<“Hello from show of second class”<<endl;
}
};
```

Output

ERROR IN THE PROGRAM

Can not access the member function of a child class with the pointer object of base class.

If you're pointing derived class using base class pointer, the pointer can not use functions defined in derived class. it can only access function derived from base class to derived class.

अगर same name के 2 functions हैं one in parent & one in child class without virtual keyword and we're using parent pointer to point child class then if we call that function using pointer it'll call function of parent class. Because it doesn't know about function defined in child class as the pointer is of parent class.

Pure virtual function and abstract class

- A pure virtual function is a function which has its body set to 0 i.e., the pure virtual function does not have any body.

- A function declared in a way as shown :

virtual void set ( ) =0;

- It simply means that the virtual function show has no body.

- The pure virtual function act as an interface and any class which inherits the class in which pure virtual function is present, has to provide the implementation for the function show.

Pure virtual function and abstract class

- **Any class which contains at least a pure virtual function is termed as an abstract class.**

- An abstract class is a class whose objects cannot be created.

- As its objects cannot be created, this class has to be inherited by some other class.

- The derived class must define the implementation of all the pure virtual function presents in the class.

Pure virtual function and abstract class

- It is not necessary that all functions in an abstract class must be pure virtual.
- An abstract class may have other virtual or non-virtual functions, data members which may be used by the objects of derived classes.
- An abstract class cannot be created but pointers and references of an abstract class can be created.
- The concept behind abstract class is to force derived classes to redefine pure virtual functions as per their requirements and features they posses.

Example 1: Pure virtual function and Abstract Class

```
#include <iostream>
using namespace std;
class Shape
{
  protected:
   float dimension;
  public:
   void getDimension()
{
      cin >> dimension;
   }
virtual float calculateArea() = 0;
};
class Square : public Shape
{
  public:
   float calculateArea()
```

Output

Enter the length of the square: 4

Area of square: 16

Enter radius of the circle: 5

Area of circle: 78.5

Example 2: Pure virtual function and Abstract Class

```
#include<iostream>
using namespace std;
class Base
{
public:
        virtual void show() = 0;
};

class Derived: public Base
{
public:
        void show()
{
cout << "In Derived \n";
}
};
```

Output

In Derived

Example 3: Pure virtual function and Abstract Class

```
#include<iostream>
using namespace std;
class Base
{
protected:
int x;
public:
virtual void fun() = 0;
Base(int i)
{
x = i;
cout<<"Constructor of base called\n";
}
};
class Derived: public Base
{
int y;
```

Output

Constructor of base called

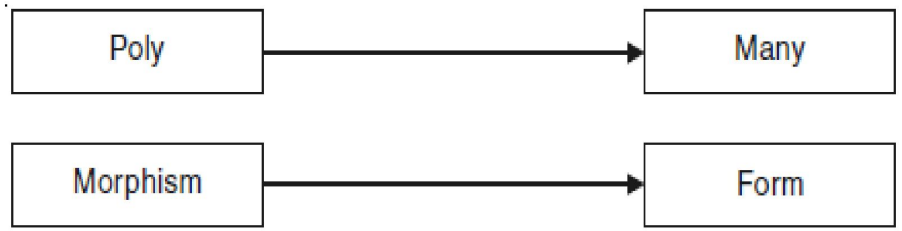
x = 4, y = 5

Constructor of base called

x = 6, y = 7

Polymorphism

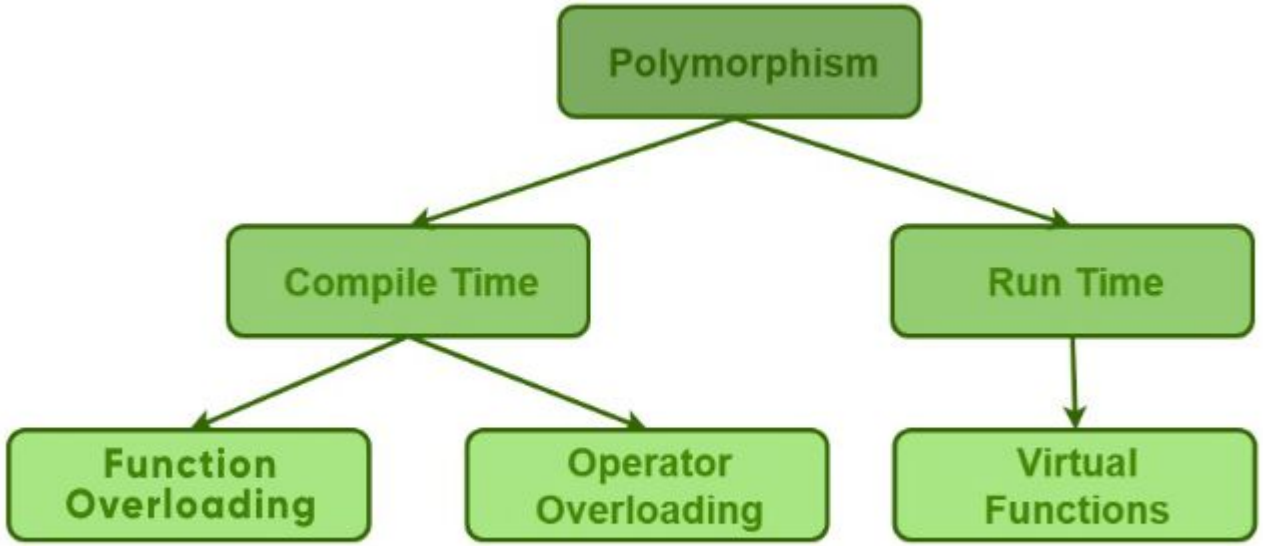
- The word Polymorphism, we get “Poly” means many and “Morphism” means form.



- Polymorphism means more than one form.
- Thus polymorphism provides a way for an entity to behave in several forms.
- An excellent example of polymorphism is Lord Krishna in Hindu methodology.

Polymorphism

- Polymorphism is considered one of the important features of Object Oriented Programming.
- Polymorphism is of two types.



Polymorphism

- It is an attribute that allows one interface to control access to a general class of actions.
- For example, if we want to find out a maximum of three numbers; no matter what type of input we pass i.e., integer, float, etc.
- The compiler automatically selects the right version of the function depending upon the type of data passed to the function max3.
- This also termed as function polymorphism or **function overloading**.

S. No	Compile Time Polymorphism	Run Time Polymorphism
1	In Compile time Polymorphism, the call is resolved by the compile time.	In Run time Polymorphism, the call is resolved at the run time.
2	It is also known as Static binding, Early binding and overloading as well.	It is also known as Dynamic binding, Late binding and overriding as well.
3	Function overloading is the compile-time polymorphism where more than one methods share the same name with different parameters or signature and different return type.	Function overriding is the runtime polymorphism having same method with same parameters or signature, but associated in different classes.
4	It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
5	It provides fast execution because the method that needs to be executed is known early at the compile time.	It provides slow execution as compare to early binding because the method that needs to be executed is known at the runtime.
6	Compile time polymorphism is less flexible as all things execute at compile time.	Run time polymorphism is more flexible as all things execute at run time.

Compile time polymorphism

- This type of polymorphism is achieved by function overloading or operator overloading.
- **Function Overloading:** Function overloading refers to creating numbers of functions with the same name which performs different tasks.
- **Operator Overloading:** C++ also provide option to overload operators. For example, we can make the operator (‘+’) for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator ‘+’ when placed between integer operands , adds them and when placed between string operands, concatenates them

Function overloading

- Function overloading is also known as function polymorphism.
- Function overloading relieves us from remembering so many functions names with type of arguments they take.
- In function overloading we can create number of functions with the same name but either number of arguments or type of arguments must be different.

int sum (int);

float sum(float);

double sum(double);

Function overloading

- Overloaded functions in a program, which function to call is determined by either checking type of argument or number of argument.
- Return type does not play any role in function overloading as which function to call is determined by checking type and number of argument a function accepts.
- When control is transferred to function and function is about to return after execution then return type comes to play.
- User defined conversion methods may be used in case a class object is to be converted to any built-in type or vice-versa.

Example 1: Demo of function overloading

```
#include <iostream.h>

void main( )

{

void show(int);

void show(float);

void show(char);

void show(char*);

int x=10;

float y=23.45;

char ch= ‘p’;

char * s=”overload”;

show(x);
```

Output

int show x =10

float show y = 23.45

char show ch = p

char *s show s = overload

Example 2: Function overloading max of two numbers

```
#include <iostream>

using namespace std;

void main( )

{

int x,y, intmax;

float f1,f2,fmax;

char ch1,ch2,chmax;

int max2(int, int);

float max2(float ,float);

char max2(char, char);

cout<<“Enter two integers \n”;

cin>>x>>y;

cout<<“Enter two floats\n”;
```

Output

Enter two integers

123 567

Enter two floats

12.34

56.78

Enter two chars

a g

Max of two int is 567

Max of two float is 56.78

Max of two char is g

Function with default arguments

- It is possible for a function not to specify all its arguments.
- Some of the arguments may be specified their default values at the time of declaring the function.
- When a function having default argument is called, compiler checks for the number and type of argument as well as was not specified during the function call, default value of that argument is assumed.
- if one argument is default, all successive arguments must be default.
- We cannot provide default values in the middle of the arguments or towards left side.

```
void display(int a, int b=0, int c=0) ✓
void display(int a=0, int b, int c=0) ✗
```

Example 1: Demo of function with default argument

```
#include <iostream>

using namespace std;

void main( )

{

void show(int x=10);

cout<<“Called show with argument 20\n”;

show(20);

cout<<“Called show without argument \n”;

show( );

}

void show(int y)
```

Output

Called show with argument 20

Argument to show was20

Called show without argument

Argument to show was10

Example 2: Demo of function with default argument

```
#include <iostream>

using namespace std;

void main( )

{

void show(int =20,int =30, int =40);

show(1,2,3);

show(1,2);

show(1);

show( );

}
```

show(1, , 3)
show(, , 3)
show(, 2,)
} All are WRONG

Output

Argument to show was

p=1 p=2 r=3

Argument to show was

p=1 p=2 r=40

Argument to show was

p=1 p=30 r=40

Argument to show was

p=20 p=30 r=40

Example 3: Demo of function with default arguments, finding bonus for an employee

```
#include <iostream>

using namespace std;

void main( )

{

void incr (float sal, float bonus_pr=10);

float s;

cout<<“Enter the salary \n”;

cin>>s;

if(s>=5000)

incr (s,15);

else
```

Output

Enter the salary

7000

Salary with bonus=8050

Function with operator overloading

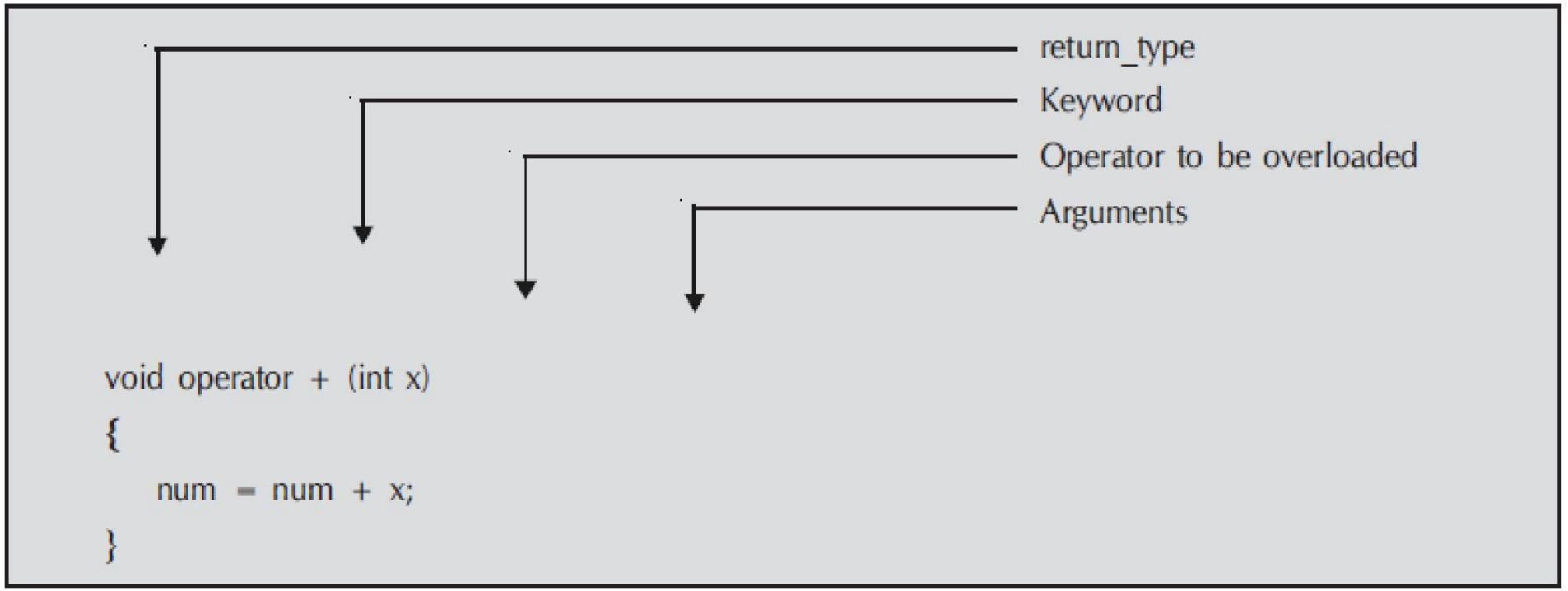
- Operator overloading refers to overloading of one operator for many different purpose.
- For example, the binary + can be used to add two integer numbers, two float numbers, two structures variables, two union variables or two class objects.
- Use of operator overloading permits us to see no difference between built-in data types and user defined data types.
- It is one of the powerful and fascinating features of the C++ which give additional meaning to built-in standard operators like **+, -, *, /, >, <, < =, > = etc.**

Function with operator overloading

- The syntax of the overloading an operator and writing new overloaded operator function is as follows :

```
return_type operator operator_symbol (arguments)
{
Statements;
Statements;
Statements;
}
```

Function with operator overloading



Example 1: Demo of operator overloading, adding integer to object

```
#include< iostream>

using namespace std;

class demo_op1

{

private :

int num;

public:

void input( )

{

cout< <"Enter the number"< <endl;

cin> > num; }

void operator + (int x)
```

Output

Enter the number

123

The num is 123

Enter the value you want to add

14

The num is 137

Rules of operator overloading

- The operators which are part of the C++ language can be overloaded. No new operator can be created using operator overloading.

- We can change the meaning of the operator i.e., a + operator can be overloaded to perform multiplication operation or > operator can be overloaded to perform addition operation. But you cannot change the priority of the operators.

- Any overloaded operator function must have at least one operand which is user defined type. All of the operands cannot be of basic types. If this is the case than function must be friend function of some class.

- In case of overloading binary operators left hand side operator must be an object of class when overloaded operator function is a member function of the class.

Rules of operator overloading

- Binary operators overloaded through member function of the class take one argument and overloaded through friend function take two arguments.
- Unary operators overloaded through member function of the class does not take any argument and overloaded through friend function must take one argument.
- There are certain operators which cannot be overloaded. They are

S.No.	*Operators*	*Name of Operator*
1.	: :	Scope resolution operator
2.	.	Dot membership operator
3.	.*	Pointer to member operator
4.	? :	Conditional operator
5.	sizeof	The size of operator

Example 2: Operator overloading with binary operator

```
#include <iostream>
using namespace std;
class demo_sum
{
private :
int num;
static int count;
public :
void input( )
{
cout<<“Enter the  number for”<<“Object”<<++count<<“\n”;
cin>>num;
}
void operator + (demo_sum temp)
{
int x;
```

Output

Enter the number forObject1

23

The num is23

Enter the number forObject2

45

The num is45

Sum of two is 68

Example 3: Overloading binary + with class objects as argument & returning objects

```
#include <iostream>
using namespace std;
class demo_sum
{
private :
int num;
static int count;
public :
void input( )
{
cout<<“ Enter the number for object”<<count++<<endl;
cin>>num;
}
demo_sum operator +(demo_sum temp)
{
demo_sum t;
```

Output

Enter the number for object1

15

Enter the number for object2

16

Enter the number for object3

18

The num is 15

The num is 16

The num is 18

The num is 49

Example 4: Overloading +, -, *, and / all in one

```
#include <iostream>
#include <stdlib>
using namespace std;
class demo_exp
{
private :
float num;
static count1, count2;
public :
void input( )
{
cout<<“Enter the number for object”<<++count1<<endl;
cin>>num;
}
demo_exp operator *(demo_exp temp)
{
```

Example 2: Overloading +, -, *, and / all in one

```
void show( )
{
cout<<“The sum for object”<<++count2<<“is”<<num<<endl;
}
};
int demo_exp : :count1;
int demo_exp : :count2;
void main( )
{
demo_exp d1,d2,d3,d4,d5;
d1.input( );
d2.input( );
d3.input( );
d4.input( );
d1.show( );
```

Output

OUTPUT :

Enter the number for object1

12

Enter the number for object2

13

Enter the number for object3

14

Enter the number for object4

15

The sum for object1is12

The sum for object2is13

The sum for object3is14

The sum for object4is15

The sum for object5 is12.166668

Example 5: Overloading pre ++ increment operator

```
#include <iostream.h>
using namespace std;
class demo
{
int num;
public :
demo( ){}
demo(int x)
{
num = x;
}
demo operator ++( )
{
demo temp;
num++;
temp.num = num;
```

Output

Output :

NUM OF OBJECT d1 :=31

NUM OF OBJECT d2 :=31

Example 6: Overloading unary - operator

```
#include <iostream.h>
using namespace std;
class demo
{
int num;
public :
demo(int x)
{
num=x;
}
void operator -( )
{
num = -num;
}
void show( )
{
```

Output

Output :

Before

num =10

After

num =-10

End of Unit III

THANKS…..